

Warehouse Delivery Robot

Intelligent Agent Modelling and Uninformed Search (BFS and DFS) on a 5×5 Grid

Problem Set-up

A delivery robot starts at cell **(1,1)** and must deliver a package to cell **(5,5)** in a 5×5 warehouse. Five cells are blocked by shelving: (2,2), (2,3), (3,3), (4,2) and (4,4). The robot moves Up, Down, Left or Right (no diagonals) and every move costs 1.

Convention used: a cell is written (row, column). Row 1 is the top row and column 1 is the left column. Up means row – 1, Down means row + 1, Left means column – 1, Right means column + 1.

	Col 1	Col 2	Col 3	Col 4	Col 5
Row 1	S (1,1)	(1,2)	(1,3)	(1,4)	(1,5)
Row 2	(2,1)	X (2,2)	X (2,3)	(2,4)	(2,5)
Row 3	(3,1)	(3,2)	X (3,3)	(3,4)	(3,5)
Row 4	(4,1)	X (4,2)	(4,3)	X (4,4)	(4,5)
Row 5	(5,1)	(5,2)	(5,3)	(5,4)	G (5,5)

S = Start, G = Goal, X = blocked (shelving unit). The grid has 25 cells, of which 5 are blocked, leaving **20 free cells** (the state space).

Part (a): PEAS Description (15 marks)

PEAS describes the task environment of an agent: Performance measure, Environment, Actuators and Sensors.

PEAS element	Description for the warehouse robot
Performance measure	• Package delivered to the correct cell (5,5): the main success criterion. • Minimum path cost: fewest moves, which also means least time and battery. • Zero collisions with shelving units or other objects. • Package delivered undamaged and without needless delay.
Environment	• A 5×5 grid warehouse with 20 free cells and 5 blocked shelving cells. • Start cell (1,1), goal cell (5,5), a single package. • Static layout: shelves do not move during the task. • Floor with grid markings, shelving, walls at the grid boundary.

PEAS element	Description for the warehouse robot
Actuators	• Drive motors / wheels to move one cell Up, Down, Left or Right. • Steering / turning mechanism. • Gripper or unloading mechanism to release the package at the goal.
Sensors	• Position sensor (wheel encoders, floor markers, QR codes or RFID) to know the current cell. • Obstacle sensors (LIDAR, ultrasonic, bumper) to detect shelving and walls. • Camera for cell identification. • Load sensor to confirm the package is held, and a battery sensor.

Task environment properties

Property	Value	Reason
Observability	Fully observable	The robot knows the whole map and its own cell.
Agents	Single agent	No other robot or opponent is involved.
Determinism	Deterministic	A move always ends in the intended cell.
Episodic or sequential	Sequential	Each move affects all later moves.
Static or dynamic	Static	Shelves stay fixed while the robot plans.
Discrete or continuous	Discrete	Finite cells and four discrete actions.
Known or unknown	Known	The effects of every action are known.

Part (b): Agent Type Classification (15 marks)

Best suited: Goal-based agent (with a model of the warehouse map).

Agent type	Suitable?	Reasoning for this scenario
Simple reflex	No	It acts on the current percept only, with no look-ahead. Cells such as (3,2) and (4,3) are dead ends, so a reflex rule like "move towards (5,5)" can trap the robot or make it loop.
Model-based reflex	Partly	It keeps an internal map and its current position, which is needed here, but it still has no explicit goal and does not plan a sequence of moves.
Goal-based	Yes	It has an explicit goal, cell (5,5). It uses the map to simulate the effect of moves and searches (BFS, DFS, A*) for a sequence of actions that reaches the goal while avoiding blocked cells and dead ends.
Utility-based	Not needed	Utility is useful when there are conflicting objectives or uncertainty (battery, congestion, risk). Here the only objective is to reach (5,5), every move costs 1, and the world is deterministic, so a goal test with a cost-minimising search is enough.

Justification. The route from (1,1) to (5,5) is a multi-step problem. The shelving creates a ring around the centre of the grid, so the robot must go round either the left side or the right side. Choosing the correct sequence needs planning (search) rather than an immediate reflex. The environment is fully observable, deterministic and static, so the agent can build the complete plan in advance from its model and then execute it. If the warehouse later had several objectives (battery use, traffic, urgent orders) the design could be upgraded to a utility-based agent.

Part (c): Formal Search Problem Formulation (15 marks)

Component	Definition
State space	$S = \{ (r, c) : 1 \leq r \leq 5, 1 \leq c \leq 5, (r, c) \notin B \}$ where $B = \{(2,2), (2,3), (3,3), (4,2), (4,4)\}$. A state is the robot's current cell, so $ S = 25 - 5 = 20$.
Initial state	$s_0 = (1,1)$
Goal state / goal test	Goal(s) is true if and only if $s = (5,5)$.
Actions	$ACTIONS(r,c)$ = a subset of $\{ \text{Up, Down, Left, Right} \}$. An action is applicable only if the resulting cell lies inside the grid and is not in B.
Transition model	$RESULT((r,c), \text{Up}) = (r - 1, c)$ $RESULT((r,c), \text{Down}) = (r + 1, c)$ $RESULT((r,c), \text{Left}) = (r, c - 1)$ $RESULT((r,c), \text{Right}) = (r, c + 1)$ Moving off the grid or into a blocked cell is not allowed (the action is not applicable).
Path cost	Step cost $c(s, a, s') = 1$ for every move. Path cost $g(n)$ = number of moves on the path from the start to n.

The same formulation written as Python code:

```

from collections import deque

N = 5
START, GOAL = (1, 1), (5, 5)
BLOCKED = {(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)}
MOVES = [("Up", (-1, 0)), ("Down", (1, 0)),
          ("Left", (0, -1)), ("Right", (0, 1))] # operator order

def successors(s):
    # ACTIONS + RESULT
    r, c = s
    for name, (dr, dc) in MOVES:
        n = (r + dr, c + dc)
        if 1 <= n[0] <= N and 1 <= n[1] <= N and n not in BLOCKED:
            yield name, n

def path_from(parent, s):
    # rebuild path from parent links
    p = []
    while s is not None:
        p.append(s); s = parent[s], return p[::-1]

```

Part (d): Breadth-First Search (25 marks)

BFS expands the **shallowest** node first using a **FIFO queue**. Neighbours are generated in the order Up, Down, Left, Right. Cells already explored or already in the queue are not added again (graph search), and the goal test is applied when a node is removed from the queue.

Algorithm (Python)

```
def bfs():
    frontier = deque([START])      # FIFO queue
    parent, explored, log = {START: None}, set(), []
    while frontier:
        s = frontier.popleft()      # remove OLDEST node
        if s == GOAL:               # goal test on removal
            log.append((s, [], list(frontier)))
            return path_from(parent, s), log
        explored.add(s)
        new = []
        for _, n in successors(s):
            if n not in explored and n not in parent:
                parent[n] = s
                frontier.append(n)   # add at the BACK
                new.append(n)
        log.append((s, new, list(frontier)))
```

Step-by-step expansion

The queue is written front → back. "Frontier after step" is the queue after the children have been added.

Step	Node expanded	Children added	Frontier (queue) after step	Notes
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)	Up and Left are outside the grid
2	(2,1)	(3,1)	(1,2), (3,1)	Up = (1,1) explored; Right = (2,2) blocked
3	(1,2)	(1,3)	(3,1), (1,3)	Down = (2,2) blocked; Left = (1,1) explored
4	(3,1)	(4,1), (3,2)	(1,3), (4,1), (3,2)	Up = (2,1) explored
5	(1,3)	(1,4)	(4,1), (3,2), (1,4)	Down = (2,3) blocked
6	(4,1)	(5,1)	(3,2), (1,4), (5,1)	Right = (4,2) blocked

Step	Node expanded	Children added	Frontier (queue) after step	Notes
7	(3,2)	none	(1,4), (5,1)	Dead end: (2,2), (4,2), (3,3) all blocked
8	(1,4)	(2,4), (1,5)	(5,1), (2,4), (1,5)	Left = (1,3) explored
9	(5,1)	(5,2)	(2,4), (1,5), (5,2)	Up = (4,1) explored
10	(2,4)	(3,4), (2,5)	(1,5), (5,2), (3,4), (2,5)	Left = (2,3) blocked
11	(1,5)	none	(5,2), (3,4), (2,5)	(2,5) is already in the queue
12	(5,2)	(5,3)	(3,4), (2,5), (5,3)	Up = (4,2) blocked
13	(3,4)	(3,5)	(2,5), (5,3), (3,5)	Down = (4,4) blocked; Left = (3,3) blocked
14	(2,5)	none	(5,3), (3,5)	(3,5) is already in the queue
15	(5,3)	(4,3), (5,4)	(3,5), (4,3), (5,4)	Both neighbours are new
16	(3,5)	(4,5)	(4,3), (5,4), (4,5)	Up and Left already explored
17	(4,3)	none	(5,4), (4,5)	Dead end: only (5,3) is open and it is explored
18	(5,4)	(5,5)	(4,5), (5,5)	Goal generated; Up = (4,4) blocked
19	(4,5)	none	(5,5)	(5,5) is already in the queue
20	(5,5)	—	(empty)	GOAL TEST SUCCEEDS

Initial frontier: [(1,1)]

Result

Following the parent links back from (5,5): $(5,5) \leftarrow (5,4) \leftarrow (5,3) \leftarrow (5,2) \leftarrow (5,1) \leftarrow (4,1) \leftarrow (3,1) \leftarrow (2,1) \leftarrow (1,1)$.

BFS path: $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

Moves: Down, Down, Down, Down, Right, Right, Right, Right

Total path cost = 8 (8 moves $\times$ cost 1). This is optimal: the Manhattan distance from (1,1) to (5,5) is 8, so no shorter path can exist.

Nodes expanded before the goal: 19. Nodes generated: 20. Maximum queue size: 4.

```
Pyt
] c
Ent File Edit Format Run Options Window Help

*untitled*

from collections import deque

BLOCKED = {(2,2), (2,3), (3,3), (4,2), (4,4)}
START, GOAL = (1,1), (5,5)

def neighbors(s):
    r, c = s
    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:      # Up, Down, Left, Right
        n = (r+dr, c+dc)
        if 1 <= n[0] <= 5 and 1 <= n[1] <= 5 and n not in BLOCKED:
            yield n

def bfs():
    queue, parent = deque([START]), {START: None}
    expanded = 0
    while queue:
        s = queue.popleft()          # FIFO
        if s == GOAL:
            break
        expanded += 1
        for n in neighbors(s):
            if n not in parent:
                parent[n] = s
                queue.append(n)
    path = []
    while s: path.append(s); s = parent[s]
    return path[::-1], expanded

path, expanded = bfs()
print(path, "cost =", len(path)-1, "expanded =", expanded)
```

```
IDLE Shell 3.12.10

File Edit Shell Debug Options Window Help

Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/manas/AppData/Local/Programs/Python/Python312/R.py =====
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)] cost = 8 expand
ed = 19
>>>
```

Part (e): Depth-First Search (20 marks)

DFS expands the **deepest** node first using a **LIFO stack**. To keep the same operator priority as BFS (Up, Down, Left, Right), children are pushed in reverse order so that Up is on top of the stack and is popped first. Explored cells are not revisited, and the goal test is applied when a node is popped.

Algorithm (Python)

```
def dfs():
    stack = [(START, None)]          # LIFO stack
    parent, explored, log = {}, set(), []
    while stack:
        s, par = stack.pop()         # remove NEWEST node
        if s in explored:
            continue
        parent[s] = par
        if s == GOAL:                # goal test on removal
            log.append((s, [], [x for x, _ in stack]))
            return path_from(parent, s), log
        explored.add(s)
        new = []
        for _, n in reversed(list(successors(s))): # reversed => Up is popped first
            if n not in explored:
                stack.append((n, s))
                new.append(n)
        log.append((s, new[::-1], [x for x, _ in stack]))
```

Step-by-step expansion

The stack is written bottom → top, so the rightmost entry is the next node to be popped.

Step	Node expanded (popped)	Children pushed	Stack after step (bottom → top)	Notes
1	(1,1)	(2,1), (1,2)	(1,2), (2,1)	Up and Left outside grid. Down (2,1) pushed last, so it is on top
2	(2,1)	(3,1)	(1,2), (3,1)	Up explored; Right = (2,2) blocked
3	(3,1)	(4,1), (3,2)	(1,2), (3,2), (4,1)	Down (4,1) is on top

Step	Node expanded (popped)	Children pushed	Stack after step (bottom → top)	Notes
4	(4,1)	(5,1)	(1,2), (3,2), (5,1)	Right = (4,2) blocked
5	(5,1)	(5,2)	(1,2), (3,2), (5,2)	Only Right is new
6	(5,2)	(5,3)	(1,2), (3,2), (5,3)	Up = (4,2) blocked
7	(5,3)	(4,3), (5,4)	(1,2), (3,2), (5,4), (4,3)	Up (4,3) is on top
8	(4,3)	none	(1,2), (3,2), (5,4)	Dead end: backtrack, pop (5,4)
9	(5,4)	(5,5)	(1,2), (3,2), (5,5)	Up = (4,4) blocked; Right = goal
10	(5,5)	—	(1,2), (3,2)	GOAL TEST SUCCEEDS

Initial stack: [(1,1)]

Result

Following the parent links back from (5,5): $(5,5) \leftarrow (5,4) \leftarrow (5,3) \leftarrow (5,2) \leftarrow (5,1) \leftarrow (4,1) \leftarrow (3,1) \leftarrow (2,1) \leftarrow (1,1)$.

DFS path: $(1,1) \rightarrow (2,1) \rightarrow (3,1) \rightarrow (4,1) \rightarrow (5,1) \rightarrow (5,2) \rightarrow (5,3) \rightarrow (5,4) \rightarrow (5,5)$

Moves: Down, Down, Down, Down, Right, Right, Right, Right

Total path cost = 8.

Nodes expanded before the goal: 9. Maximum stack size: 4. The nodes (1,2) and (3,2) were still on the stack, unexplored, when the goal was found.

Note: DFS found a cost-8 path here because, with Down having higher priority than Right, it went straight down column 1 and then along row 5. DFS does not guarantee this. The dead end at (4,3) shows how DFS goes deep, backtracks, and may take a longer route if the operator order is different.

```
*untitled*
File Edit Format Run Options Window Help
BLOCKED = {(2,2), (2,3), (3,3), (4,2), (4,4)}
START, GOAL = (1,1), (5,5)

def neighbors(s):
    r, c = s
    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]: # Up, Down, Left, Right
        n = (r+dr, c+dc)
        if 1 <= n[0] <= 5 and 1 <= n[1] <= 5 and n not in BLOCKED:
            yield n

def dfs():
    stack, parent = [(START, None)], {}
    expanded = 0
    while stack:
        s, p = stack.pop() # LIFO
        if s in parent:
            continue
        parent[s] = p
        if s == GOAL:
            break
        expanded += 1
        for n in reversed(list(neighbors(s))): # Up is popped first
            if n not in parent:
                stack.append((n, s))
    path = []
    while s: path.append(s); s = parent[s]
    return path[::-1], expanded

path, expanded = dfs()
print(path, "cost =", len(path)-1, "expanded =", expanded)
```

```
===== RESTART: C:/Users/manas/AppData/Local/Programs/Python/Python312/RM.py =====
[(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)] cost = 8 expanded = 9
```

Part (f): Comparison of BFS and DFS (10 marks)

Notation: b = branching factor (at most 4 here), d = depth of the shallowest goal (8 here), m = maximum depth of the search space, $|S| = 20$ states.

Criterion	BFS	DFS	This grid
Completeness	Complete: finite b and a finite state space, so it always finds the goal if one exists.	Complete only with an explored set on a finite graph. Plain tree-search DFS can loop forever (for example Up, Down, Up, Down).	Both complete
Optimality	Optimal when every step cost is equal (here, 1): it returns the shallowest goal.	Not optimal in general: it returns the first goal found, which may be deeper.	Both found cost 8
Time complexity	$O(b^d)$. With an explored set, bounded by $O(S)$.	$O(b^m)$. With an explored set, bounded by $O(S)$.	BFS 19 expansions, DFS 9
Space complexity	$O(b^d)$: the whole frontier is stored, which is the main weakness.	$O(b \cdot m)$: only the current path and its siblings are stored.	Peak size 4 for both

Which is more suitable?

BFS is more suitable for this warehouse delivery problem, for these reasons:

- **Every move costs 1**, so BFS is guaranteed to return the cheapest path (8 moves). A delivery robot should save time and battery, so a guarantee of optimality matters.
- **The state space is tiny** (20 cells). The usual disadvantage of BFS, its memory use, is negligible: the queue never held more than 4 nodes.
- **DFS was only lucky**. It needed fewer expansions (9 against 19) and found the same cost here, but that depended on the operator order. With a different order, a loop, or a larger layout with more detours, DFS could return a much longer route, and nothing in the algorithm warns about it.

- **Predictable behaviour.** BFS always gives the same shortest answer whatever the operator order, which is important for a safety-critical, repeatable robot task.

DFS would be preferred only if memory were severely limited and any valid path was acceptable. For a much larger warehouse, A* search with the Manhattan-distance heuristic would be a better choice than either, because it keeps optimality while expanding far fewer nodes.

CONCLUSION:

The warehouse delivery robot problem demonstrates the essential transition from reactive reflexes to deliberate, goal-oriented decision-making in discrete, obstacle-dense environments. Because the operational space includes concave obstacle clusters that trap simple reflex mechanisms, formulating the task as a formal search problem ensures that every potential action sequence is vetted against the goal state before physical execution.

Empirical and theoretical comparisons highlight Breadth-First Search (BFS) as the distinctly superior algorithm for this task. In automated logistics, route predictability and path optimality directly dictate battery consumption, delivery cycle times, and operational wear. Because all orthogonal steps carry a uniform cost of 1, BFS provides an absolute mathematical guarantee of finding the shortest path (8 moves). While Depth-First Search (DFS) reached the destination in fewer node expansions due to favorable directional tie-breaking toward the goal, it offers no intrinsic guarantee of optimality and remains vulnerable to severe detours in more complex layouts.

.